

```

template<typename T>                                //声明 helper template
const Rational<T> doMultiply(const Rational<T>& lhs,
                             const Rational<T>& rhs);

template<typename T>
class Rational {
public:
    ...
friend
    const Rational<T> operator*(const Rational<T>& lhs,
                                const Rational<T>& rhs)
    { return doMultiply(lhs, rhs); }                //令 friend 调用 helper
    ...
};

```

许多编译器实质上会强迫你把所有 `template` 定义式放进头文件内，所以你或许需要在头文件内定义 `doMultiply`（一如条款 30 所言，这样的 `templates` 不需非得是 `inline` 不可），看起来像这样：

```

template<typename T>                                //若有必要，
const Rational<T> doMultiply(const Rational<T>& lhs, //在头文件内定义
                             const Rational<T>& rhs) //helper template
{
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}

```

作为一个 `template`，`doMultiply` 当然不支持混合式乘法，但它其实也不需要。它只被 `operator*` 调用，而 `operator*` 支持了混合式操作！本质上 `operator*` 支持了类型转换所需的任何东西，确保两个 `Rational` 对象能够相乘，然后它将这两个对象传给一个适当的 `doMultiply` `template` 具现体，完成实际的乘法操作。协作为成功之本，不是吗？

请记住

- 当我们编写一个 `class template`，而它所提供之“与此 `template` 相关的”函数支持“所有参数之隐式类型转换”时，请将那些函数定义为“`class template` 内部的 `friend` 函数”。

条款 47：请使用 `traits classes` 表现类型信息

Use traits classes for information about types.

STL 主要由“用以表现容器、迭代器和算法”的 `templates` 构成，但也覆盖若干工具性 `templates`，其中一个名为 `advance`，用来将某个迭代器移动某个给定距离：

```
template<typename IterT, typename DistT> //将迭代器向前移动 d 单位。  
void advance(IterT& iter, DistT d); //如果 d < 0 则向后移动。
```

观念上 `advance` 只是做 `iter += d` 动作, 但其实不可以全然那么实践, 因为只有 **random access** (随机访问) 迭代器才支持 `+=` 操作。面对其他威力不那么强大的迭代器种类, `advance` 必须反复施行 `++` 或 `--`, 共 `d` 次。

嗯, 你不记得你的 STL 迭代器分类 (categories) 了吗? 没关系, 让我们来一次迷你回顾。STL 共有 5 种迭代器分类, 对应于它们支持的操作。**Input** 迭代器只能向前移动, 一次一步, 客户只可读取 (不能涂写) 它们所指的东, 而且只能读取一次。它们模仿指向输入文件的阅读指针 (read pointer); C++ 程序库中的 `istream_iterators` 是这一分类的代表。**Output** 迭代器情况类似, 但一切只为输出: 它们只向前移动, 一次一步, 客户只可涂写它们所指的东, 而且只能涂写一次。它们模仿指向输出文件的涂写指针 (write pointer); `ostream_iterators` 是这一分类的代表。这是威力最小的两个迭代器分类。由于这两类都只能向前移动, 而且只能读或写其所指物最多一次, 所以它们只适合“一次性操作算法” (one-pass algorithms)。

另一个威力比较强大的分类是 **forward** 迭代器。这种迭代器可以做前述两种分类所能做的每一件事, 而且可以读或写其所指物一次以上。这使得它们可施行于多次性操作算法 (multi-pass algorithms)。STL 并未提供单向 linked list, 但某些程序库有 (通常名为 `slist`), 而指入这种容器的迭代器就是属于 **forward** 迭代器。指入 TR1 **hashed** 容器 (见条款 54) 的也可能是这一分类。(译注: 这里说“可能”是因为 **hashed** 容器的迭代器可为单向也可为双向, 取决于实现版本。)

Bidirectional 迭代器比上一个分类威力更大: 它除了可以向前移动, 还可以向后移动。STL 的 `list` 迭代器就属于这一分类, `set`, `multiset`, `map` 和 `multimap` 的迭代器也都是这一分类。

最有威力的迭代器当属 **random access** 迭代器。这种迭代器比上一个分类威力更大的地方在于它可以执行“迭代器算术”, 也就是它可以在常量时间内向前或向后跳跃任意距离。这样的算术很类似指针算术, 那并不令人惊讶, 因为 **random access** 迭代器正是以内置 (原始) 指针为榜样, 而内置指针也可被当做 **random access** 迭代器使用。`vector`, `deque` 和 `string` 提供的迭代器都是这一分类。

对于这 5 种分类, C++ 标准程序库分别提供专属的卷标结构 (tag struct) 加以确认:

```

struct input_iterator_tag {};
struct output_iterator_tag {};
struct forward_iterator_tag: public input_iterator_tag { };
struct bidirectional_iterator_tag: public forward_iterator_tag { };
struct random_access_iterator_tag: public bidirectional_iterator_tag {};

```

这些 `structs` 之间的继承关系是有效的 *is-a* 关系(见条款 32): 是的, 所有 *forward* 迭代器都是 *input* 迭代器, 依此类推。很快我们会看到这个继承关系的效力。

现在回到 `advance` 函数。我们已经知道 STL 迭代器有着不同的能力, 实现 `advance` 的策略之一是采用“最低但最普及”的迭代器能力, 以循环反复递增或递减迭代器。然而这种做法耗费线性时间。我们知道 *random access* 迭代器支持迭代器算术运算, 只耗费常量时间, 因此如果面对这种迭代器, 我们希望运用其优势。

我们真正希望的是以这种方式实现 `advance`:

```

template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    if (iter is a random access iterator) {
        iter += d;        //针对 random access 迭代器使用迭代器算术运算
    }
    else {
        if (d >= 0) { while (d--) ++iter; }    //针对其他迭代器分类
        else { while (d++) --iter; }          //反复调用 ++ 或 --
    }
}

```

这种做法首先必须判断 `iter` 是否为 *random access* 迭代器, 也就是说需要知道类型 `IterT` 是否为 *random access* 迭代器分类。换句话说我们需要取得类型的某些信息。那就是 *traits* 让你得以进行的事: 它们允许你在编译期间取得某些类型信息。

Traits 并不是 C++ 关键字或一个预先定义好的构件; 它们是一种技术, 也是一个 C++ 程序员共同遵守的协议。这个技术的要求之一是, 它对内置 (built-in) 类型和用户自定义 (user-defined) 类型的表现必须一样好。举个例子, 如果上述 `advance` 收到的实参是一个指针 (例如 `const char*`) 和一个 `int`, 上述 `advance` 仍然必须有效运作, 那意味 *traits* 技术必须也能够施行于内置类型如指针身上。

“*traits* 必须能够施行于内置类型”意味“类型内的嵌套信息 (nesting information)”这种东西出局了, 因为我们无法将信息嵌套于原始指针内。因此类型的 *traits* 信息必须位于类型自身之外。标准技术是把它放进一个 `template` 及其一或多个特化版本中。这样的 `templates` 在标准程序库中有若干个, 其中针对迭代器者被命名为 `iterator_traits`:

```
template<typename IterT>           //template, 用来处理
struct iterator_traits;           //迭代器分类的相关信息
```

如你所见, `iterator_traits` 是个 `struct`。是的, 习惯上 `traits` 总是被实现为 `structs`, 但它们却又往往被称为 `traits classes`。

`iterator_traits` 的运作方式是, 针对每一个类型 `IterT`, 在 `struct iterator_traits<IterT>` 内一定声明某个 `typedef` 名为 `iterator_category`。这个 `typedef` 用来确认 `IterT` 的迭代器分类。

`iterator_traits` 以两个部分实现上述所言。首先它要求每一个“用户自定义的迭代器类型”必须嵌套一个 `typedef`, 名为 `iterator_category`, 用来确认适当的卷标结构 (tag struct)。例如 `deque` 的迭代器可随机访问, 所以一个针对 `deque` 迭代器而设计的 `class` 看起来会是这样子:

```
template < ... >                 //略而未写 template 参数
class deque {
public:
    class iterator {
    public:
        typedef random_access_iterator_tag iterator_category;
        ...
    };
    ...
};
```

`list` 的迭代器可双向行进, 所以它们应该是这样:

```
template < ... >
class list {
public:
    class iterator {
    public:
        typedef bidirectional_iterator_tag iterator_category;
        ...
    };
    ...
};
```

至于 `iterator_traits`, 只是鹦鹉学舌般地响应 `iterator class` 的嵌套式 `typedef`:

```
//类型 IterT 的 iterator_category 其实就是用来表现 “IterT 说它自己是什么”。
//关于 “typedef typename” 的运用, 见条款 42。
template<typename IterT>
struct iterator_traits {
    typedef typename IterT::iterator_category iterator_category;
    ...
};
```

这对用户自定义类型行得通，但对指针（也是一种迭代器）行不通，因为指针不可能嵌套 `typedef`。 `iterator_traits` 的第二部分如下，专门用来对付指针。

为了支持指针迭代器， `iterator_traits` 特别针对指针类型提供一个偏特化版本（*partial template specialization*）。由于指针的行径与 *random access* 迭代器类似，所以 `iterator_traits` 为指针指定的迭代器类型是：

```
template<typename IterT>                //template 偏特化
struct iterator_traits<IterT*>          //针对内置指针
{
    typedef random_access_iterator_tag iterator_category;
    ...
};
```

现在，你应该知道如何设计并实现一个 `traits class` 了：

- 确认若干你希望将来可取得的类型相关信息。例如对迭代器而言，我们希望将来可取得其分类（`category`）。
- 为该信息选择一个名称（例如 `iterator_category`）。
- 提供一个 `template` 和一组特化版本（例如稍早说的 `iterator_traits`），内含你希望支持的类型相关信息。

好，现在有了 `iterator_traits`（实际上是 `std::iterator_traits`，因为它是 C++ 标准程序库的一部分），我们可以对 `advance` 实践先前的伪码（*pseudocode*）：

```
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    if (typeid(typename std::iterator_traits<IterT>::iterator_category)
        ==typeid(std::random_access_iterator_tag))
        ...
}
```

虽然这看起来前景光明，其实并非我们想要。首先它会导致编译问题，但我将在条款 48 才探讨这一点，此刻有更根本的问题要考虑。`IterT` 类型在编译期间获知，所以 `iterator_traits<IterT>::iterator_category` 也可在编译期间确定。但 `if` 语句却是在运行期才会核定。为什么将可在编译期完成的事延到运行期才做呢？这不仅浪费时间，也造成可执行文件膨胀。

我们真正想要的是一个条件式（也就是一个 `if...else` 语句）判断“编译期核定成功”之类型。恰巧 C++ 有一个取得这种行为的办法，那就是重载（*overloading*）。

当你重载某个函数 `f`，你必须详细叙述各个重载件的参数类型。当你调用 `f`，编译器便根据传来的实参选择最适当的重载件。编译器的态度是“如果这个重载件最匹配传递过来的实参，就调用这个 `f`；如果那个重载件最匹配，就调用那个 `f`；如果第三个 `f` 最匹配，就调用第三个 `f`！”依此类推。看到了吗，这正是一个针对类型而发生的“编译期条件句”。为了让 `advance` 的行为如我们所期望，我们需要做的是产生两版重载函数，内含 `advance` 的本质内容，但各自接受不同类型的 `iterator_category` 对象。我将这两个函数取名为 `doAdvance`：

```
template<typename IterT, typename DistT>           //这份实现用于
void doAdvance(IterT& iter, DistT d,              //random access
               std::random_access_iterator_tag) //迭代器
{
    iter += d;
}

template<typename IterT, typename DistT>           //这份实现用于
void doAdvance(IterT& iter, DistT d,              //bidirectional
               std::bidirectional_iterator_tag) //迭代器
{
    if (d >= 0) { while (d--) ++iter; }
    else { while (d++) --iter; }
}

template<typename IterT, typename DistT>           //这份实现用于
void doAdvance(IterT& iter, DistT d,              //input 迭代器
               std::input_iterator_tag)
{
    if (d < 0 ) {
        throw std::out_of_range("Negative distance"); //详下
    }
    while (d--) ++iter;
}
```

由于 `forward_iterator_tag` 继承自 `input_iterator_tag`，所以上述 `doAdvance` 的 `input_iterator_tag` 版本也能够处理 *forward* 迭代器。这是 `iterator_tag structs` 继承关系带来的一项红利。实际上这也是 `public` 继承带来的部分好处：针对 `base class` 编写的代码用于 `derived class` 身上也行得通。

`advance` 函数规范说，如果面对的是 *random access* 和 *bidirectional* 迭代器，则接受正距离和负距离；但如果面对的是 *forward* 或 *input* 迭代器，则移动负距离会导致不明确（未定义）行为。我所检验过的实现码都假设 `d` 不为负，于是直接进入一个冗长的循环迭代，等待计数器降为 0。上述代码中我以抛出异常取而代之。两种做法都有根据，但“无法预言发生何事”是“不明确行为”之祸源所在。

有了这些 `doAdvance` 重载版本, `advance` 需要做的只是调用它们并额外传递一个对象, 后者必须带有适当的迭代器分类。于是编译器运用重载解析机制 (overloading resolution) 调用适当的实现代码:

```
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    doAdvance(                //调用的 doAdvance 版本
        iter, d,              //对 iter 之迭代器分类而言
        typename              //必须是适当的。
        std::iterator_traits<IterT>::iterator_category()
    );
}
```

现在我们可以总结如何使用一个 `traits class` 了:

- 建立一组重载函数 (身份像劳工) 或函数模板 (例如 `doAdvance`), 彼此间的差异只在于各自的 `traits` 参数。令每个函数实现码与其接受之 `traits` 信息相应和。
- 建立一个控制函数 (身份像工头) 或函数模板 (例如 `advance`), 它调用上述那些“劳工函数”并传递 `traits class` 所提供的信息。

`Traits` 广泛用于标准程序库。其中当然有上述讨论的 `iterator_traits`, 除了供应 `iterator_category` 还供应另四份迭代器相关信息 (其中最有用的是 `value_type`, 见条款 42)。此外还有 `char_traits` 用来保存字符类型的相关信息, 以及 `numeric_limits` 用来保存数值类型的相关信息, 例如某数值类型可表现之最小值和最大值等等; 命名为 `numeric_limits` 有点让人惊讶, 因为 `traits classes` 的名称常以 “traits” 结束, 但 `numeric_limits` 却没有遵守这种风格。

TR1 (条款 54) 导入许多新的 `traits classes` 用以提供类型信息, 包括 `is_fundamental<T>` (判断 `T` 是否为内置类型), `is_array<T>` (判断 `T` 是否为数组类型), 以及 `is_base_of<T1, T2>` (`T1` 和 `T2` 相同, 抑或 `T1` 是 `T2` 的 `base class`)。总计 TR1 一共为标准 C++ 添加了 50 个以上的 `traits classes`。

请记住

- `Traits classes` 使得“类型相关信息”在编译期可用。它们以 `templates` 和“`templates` 特化”完成实现。
- 整合重载技术 (overloading) 后, `traits classes` 有可能在编译期对类型执行 `if...else` 测试。

条款 48: 认识 template 元编程

Be aware of template metaprogramming.

Template metaprogramming (TMP, 模板元编程) 是编写 template-based C++ 程序并执行于编译期的过程。花一分钟想想这个: 所谓 template metaprogram (模板元程序) 是以 C++ 写成、执行于 C++ 编译器内的程序。一旦 TMP 程序结束执行, 其输出, 也就是从 templates 具现出来的若干 C++ 源码, 便会一如往常地被编译。

如果这没有带给你异乎寻常的印象, 你一定没有足够认真地思考它。

C++ 并非是为 template metaprogramming 而设计, 但自从 TMP 于 1990s 初期被发现以后, 由于日渐被证明十分有用, 其延伸部分很可能加入语言和标准程序库内, 使 TMP 更容易进行。是的, TMP 是被发现而不是被发明出来的。当 templates 加入 C++ 时 TMP 底层特性也就被引进了。对某些人而言唯一需要注意的是如何以熟练巧妙而意想不到的方式使用 TMP。

TMP 有两个伟大的效力。第一, 它让某些事情更容易。如果没有它, 那些事情将是困难的, 甚至不可能的。第二, 由于 template metaprograms 执行于 C++ 编译期, 因此可将工作从运行期转移到编译期。这导致的一个结果是, 某些错误原本通常在运行期才能侦测到, 现在可在编译期找出来。另一个结果是, 使用 TMP 的 C++ 程序可能在每一方面都更高效: 较小的可执行文件、较短的运行期、较少的内存需求。然而将工作从运行期移转至编译期的另一个结果是, 编译时间变长了。是的, 程序如果使用 TMP, 其编译时间可能远长于不使用 TMP 的对应版本。

考虑 p.228 导入的 STL advance 伪码(位于条款 47。或许你会想现在就阅读它, 因为本条款中我假设你已经熟悉条款 47 的内容)。就像 p.228 所示, 我特别强调那段代码的伪码部分 (pseudo part):

```
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    if (iter is a random access iterator) {
        iter += d;    //针对 random access 迭代器使用迭代器算术运算
    }
    else {
        if (d >= 0) { while (d--) ++iter; }    //针对其他迭代器类型
        else { while (d++) --iter; }          //反复调用 ++ 或 --
    }
}
```